

Autonome Eingebettete Systeme (AEiSy)

Level 8 – Odometer

1. Odometrie
2. Messprinzip
3. Technische Daten
4. Schnittstellen
5. Aufgabe

Level 8: Odometer

Lernziele:

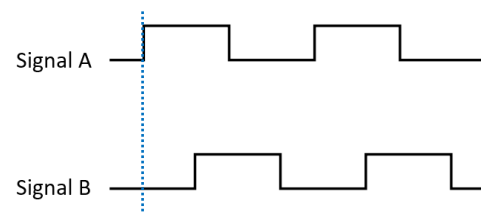
- Inbetriebnahme der Radencoder
→ Auswertung der zurückgelegten Strecke ⇨ Odometer
- Inbetriebnahme der notwendigen Peripherie des STM32G474 (GPIO, Interrupts)
- Design, Entwicklung und Test eines SW-Moduls in Abhängigkeit der Anforderungen und Randbedingungen
- Integration in bestehendes Embedded SW-System

1. Odometrie

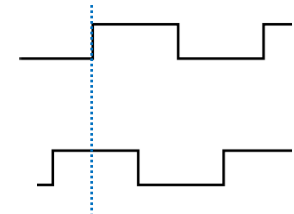
- Wegmessung → Odometer = Kilometerzähler /
Messung der zurückgelegten Distanz
→ Tachometer = Geschwindigkeitsmesser
- Motivation: präzisere Messung der zurückgelegten Entfernung
als im Vergleich zur zeitlichen Steuerung der Motoren

2. Messprinzip

- Inkrementalgeber / Drehencoder
 - Umwandlung einer Drehbewegung in ein elektrisches Signal
 - Weg-/Winkelmessung, Geschwindigkeitsmessung
- Photoelektrische Abtastung (Durchlichtverfahren):
 - lichtdurchlässige Schlitze am Rand einer Segmentscheibe (→ Inkremente)
 - Einweglichtschranke mit zwei leicht versetzten Photodetektoren (Phototransistoren)
- Ausgangssignale:
 - Signal A
 - Signal B



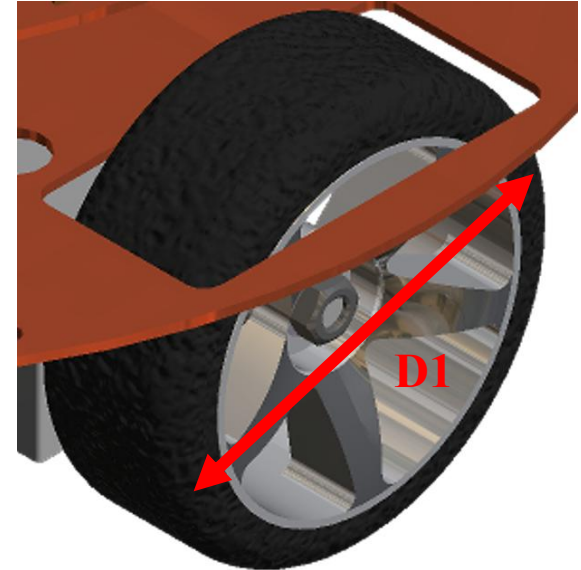
Drehrichtung: gegen den Uhrzeigersinn
(Sichtweise Lichtdetektor)



Drehrichtung im Uhrzeigersinn

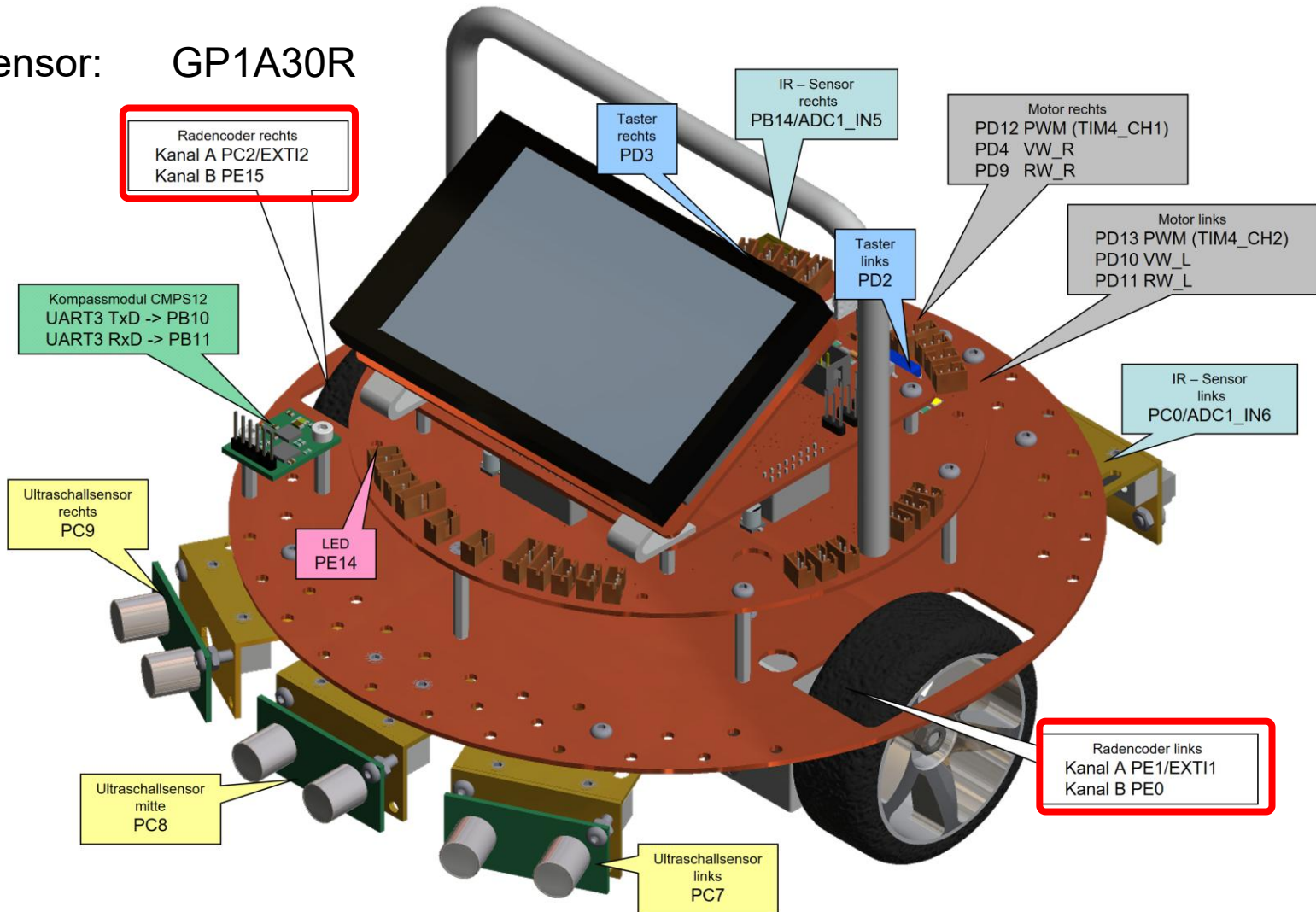
3. Technische Daten

- Radencoder: GP1A30R
- Auflösung eines Vollkreises:
Anzahl der hell-dunkel Felder entlang des
Umfangs der Segmentscheibe
⇒ hier: 120 Inc / Umdrehung
- Raddurchmesser: $D1 = 66 \text{ mm}$
- Radabstand: $D2 = 175 \text{ mm}$



4. Schnittstellen im MCRobot

- Sensor: GP1A30R



5. Aufgaben Optional!

5.1 Moduldesign

- Definition der Schnittstellen zur Hardware (z.B. GPIO und Interrupts) und übergeordneter SW-Schichten.
- Feindesign der Funktion „Odometer/Encoder“:
 - Definition der SW-Module (.c/.h Dateien) (z.B. gpio.c/h / encoder.c/h,...)
 - Definition der SW-Schnittstellen (Funktionen) vgl. Level 7:
 - Drive Forward (Distance)
 - Drive Backward (Distance)
 - ...

„Drive Control“

„Odo-
meter“

L8

Hardware

5. Aufgaben

5.2 Implementierung / Integration

- Code-Entwicklung der Funktionen (5.1)

→ EXTI - Interrupts + Handler zum Zählen der Inkremente:

C-Signatur: `void EXTI1_IRQHandler(void)` (links)
 `void EXTI2_IRQHandler(void)` (rechts)

→ Aktivierung der Interrupts über CMSIS-Funktionen:

`NVIC_EnableIRQ(EXTI1_IRQn);`

→ Konfiguration der externen Interrupts (STM32G474 Dokumentation;
RM0440, Kapitel 15; Register: `RCC->APB2ENR`,
`SYSCFG->EXTICR`, `EXTI->RTSR1` (`FTSR1`), `EXTI->IMR1`)

Hinweis: Die Pins müssen als „Input“
konfiguriert werden → `RCC->AHB2ENR`, `GPIOC->MODER`,
 `GPIOE->MODER`
`APB2ENR` → für `SYSCFGEN`

5. Aufgaben

5.2 Implementierung / Integration

- **Einfach:** (fehleranfällig)

Die steigende Flanke an Kanal A löst einen Interrupt aus, der Pegel von Kanal B gibt im Handler dann das Vorzeichen der Zähleränderung vor.

- **Genauer:**

Beide Flanken an Kanal A **und** Kanal B lösen Interrupts aus; im Handler wird der neue Pin-Zustand {A,B} gelesen und zusammen mit dem zuletzt gespeicherten Vorzustand als Index auf eine Übergangstabelle angewendet, die für jeden der acht gültigen Ein-Bit-Übergänge ein +1 oder –1 liefert. Zwei-Bit-Sprünge (durch Prellen oder verpasste Flanken) ergeben gezielt 0 und werden so verworfen.

```
static const int8_t lut[16] = {  
    //          new=00  new=01  new=10  new=11  
    /* prev = 00 */    0,      -1,      +1,      0,  
    /* prev = 01 */    +1,      0,      0,      -1,  
    /* prev = 10 */    -1,      0,      0,      +1,  
    /* prev = 11 */    0,      +1,      -1,      0  
};
```

```
uint32_t status_neu = GPIOE->IDR & 0x03u;          //liegen an PortE bereits in 0...3 Kodierung  
encoder_counter += lut[(status_alt << 2) | status_neu]; // x*4 + (0...3)  
status_alt = (uint8_t)status_neu;
```